Mārcis Deguts md24031

MD_Mem apraksts

Problēmas un tehniskas detaļas

Uzdevuma mērķis bija salīdzināt trīs atmiņas rezervācijas veidus: **malloc, mmap un sbrk**. Programmā md_mem.c, kur bija jānosaka maksimālais rezervējamais atmiņas daudzums, rezervējot atmiņu pa 1 MB. Lai atvieglotu dzīvi, es izveidoju programmas tā, lai rezervācijas veidu var izvēlēties caur komandrindas argumentu. Galvenā problēma, ar kuru saskāros, bija tā, ka sākotnējā pieeja, kur bezgalīgā ciklā katrā iterācijā tika rezervēts vēl viens 1 MB bloks, nebija praktiska. Ja rezervētā atmiņa tika arī reāli izmantota, Linux sistēma pie atmiņas trūkuma vienkārši pārtrauca procesu ar OOM mehānismu, un gala rezultātu vairs nevarēja korekti nolasīt. Tāpēc es pamainīju pieeju. Tā vietā, lai uzkrātu vairākus 1 MB bloku vienlaikus, es katrā cikla iterācijā rezervēju tikai vienu atmiņas bloku, kura izmērs pakāpeniski pieauga par 1 MB. Ja rezervācija izdevās, bloks tika atbrīvots un nākamajā iterācijā tika mēģināts rezervēt jau lielāku bloku (N->N+1). Šāda pieeja ļāva noteikt lielāko vienā piegājienā rezervējamo atmiņas daudzumu, neuzkrājot visus iepriekšējos blokus un neizraisot procesa nogalināšanu.

Ja rezervētajai atmiņai netiek piekļūts, operētājsistēma bieži piešķir tikai virtuālo adrešu telpu, un tādēļ iespējams rezervēt daudz lielāku apjomu nekā ir pieejams fiziskajā RAM. Savukārt, ja rezervētā atmiņa tiek aizpildīta, piemēram, ar memset, sistēmai jāpiešķir arī reālas fiziskās lapas. Šī iemsla dēļ, es md_mem programmā neaizpildīju ar memset to atmiņu, jo tad process kļuva ĻOTI lēns.

Testēšana

Lai atvieglotu testēšanu, es rezultātus glabāju teksta failos, un automatizēju testēšanu ar Makefile un vienu bash skriptu. Bash skriptu es uztaisīju, lai dabūtu vidējo laiku priekš md_mem_100. Es arī serverī augšupielādēju bash skriptu un Makefile.

```
md@fedora:~/Desktop/LSP/md/mem$ make all
gcc -o md_mem md_mem.c
gcc -o md_mem_100 md_mem_100.c
md@fedora:~/Desktop/LSP/md/mem$ make test_time_bash
./test_time.sh >> time.txt
md@fedora:~/Desktop/LSP/md/mem$ make test_size
objdump -h md_mem_100 >> size.txt
echo "SIZE INFO" >> size.txt
size md_mem_100 >> size.txt
md@fedora:~/Desktop/LSP/md/mem$ make test_mem
./md_mem malloc >> mem.txt
echo >> mem.txt
./md_mem sbrk >> mem.txt
echo >> mem.txt
./md_mem mmap >> mem.txt
echo >> mem.txt
md@fedora:~/Desktop/LSP/md/mem$
```

Mārcis Deguts md24031

Rezultāti

Maksimālais atmiņas daudzums

```
Results with method: malloc
Total memory allocated(bytes): 25052577792
Total MB allocated: 23892

Results with method: sbrk
Total memory allocated(bytes): 25052577792
Total MB allocated: 23892

Results with method: mmap
Total memory allocated(bytes): 25052577792
Total MB allocated: 23892
```

Var redzēt, ka ka visām trim rezervācijas maksimāli rezervējamais atmiņas apjoms bija praktiski vienāds (aptuveni **23892 MB~23GB)**. Tas ir tāpēc, ka šajā testā atmiņa tika tikai rezervēta, bet netika reāli izmantota, tādēļ operētājsistēma piešķīra tikai virtuālo adreses telpu. **Maksimālo rezervējamo apjomu šajā gadījumā vairāk nosaka sistēmas virtuālās atmiņas limiti, nevis būtiskas atšķirības starp pašām rezervācijas funkcijām**. Tāpēc šie skaitļi **var ļoti atšķirties** atkarībā no datora uz kura tiek palaista programma. Piemēram, man uz cita datora ar WSL (Windows subsystem for Linux), rezultāts bija ap ~9.7GB.

```
marcis@DESKTOP-JDJNPJU:~/Projects/Linux-Programming/md/mem$ make test_mem
Allocted MB thus far: 5376
Allocted MB thus far: 5632
Allocted MB thus far: 5888
Allocted MB thus far: 6144
Allocted MB thus far: 6400
Allocted MB thus far: 6656
Allocted MB thus far: 6912
Allocted MB thus far: 7168
Allocted MB thus far: 7424
Allocted MB thus far: 7680
Allocted MB thus far: 7936
Allocted MB thus far: 8192
Allocted MB thus far: 8448
Allocted MB thus far: 8704
Allocted MB thus far: 8960
Allocted MB thus far: 9216
Allocted MB thus far: 9472
Allocted MB thus far: 9728
echo >> mem.txt
marcis@DESKTOP-JDJNPJU:~/Projects/Linux-Programming/md/mem$
```

Mārcis Deguts md24031

Darbības laiks

```
Results with method(100MB): malloc
With writing? (touch): Enabled
Total memory allocated(bytes): 104857600
Total MB allocated: 100

real    0m0.062s
user    0m0.007s
sys 0m0.054s

Results with method(100MB): sbrk
With writing? (touch): Enabled
Total memory allocated(bytes): 104857600
Total MB allocated: 100

real    0m0.057s
user    0m0.007s
sys 0m0.050s

Results with method(100MB): mmap
With writing? (touch): Enabled
Total memory allocated(bytes): 104857600
Total MB allocated: 100

real    0m0.056s
user    0m0.007s
sys 0m0.048s
```

Kad tika rezervēta atmiņas, tad pie reizes arī programmā ar memset tika izmantota tā atmiņa. Katram veidam programma tika izpildīta 10 reizes un iegūts vidējais laiks ar Bash skriptu. Var redzēt, ka visu trīs rezervācijas metožu darbības laiki bija ļoti līdzīgi. Vidējais izpildes laiks, ņemot vērā user un sys laiku summu, bija aptuveni **0.061 s malloc, 0.057 s sbrk un 0.055 s mmap**. No šiem rezultātiem redzams, ka visātrākā metode konkrēti priekš 100MB bija mmap, kam sekoja sbrk, bet nedaudz lēnāks bija malloc. Tas var tikt skaidrots ar to, ka malloc ir bibliotēkas līmeņa funkcija, kas papildus pašai rezervācijai veic arī atmiņas pārvaldību, piemēram, tiek uzturēti metadati, un arī malloc pats izmanto mmap. Savukārt mmap un sbrk ir tiešāki sistēmas izsaukumi. Tomēr starpība starp metodēm bija ļoti neliela, tāpēc šāda izmēra atmiņas rezervēšanas gadījumā rezervācijas veida izvēle būtiski neietekmē programmas izpildes laiku.

Mārcis Deguts md24031

Statiskās atmiņas izmēri

```
md_mem_100:     file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .note.gnu.build-id 00000024  0000000000400318  0000000000400318  00000318  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .init         0000001b  000000000040033c  000000000040033c  0000033c  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  2 .plt          00000080  0000000000400360  0000000000400360  00000360  2**4
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  3 .text         00000380  00000000004003e0  00000000004003e0  000003e0  2**4
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  4 .fini         0000000d  0000000000400760  0000000000400760  00000760  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  5 .interp       0000001c  0000000000401000  0000000000401000  00001000  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .gnu.hash     00000024  0000000000401020  0000000000401020  00001020  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .dynsym       00000108  0000000000401048  0000000000401048  00001048  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 .dynstr       00000071  0000000000401150  0000000000401150  00001150  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 .gnu.version  00000016  00000000004011c2  00000000004011c2  000011c2  2**1
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 10 .gnu.version_r 00000030  00000000004011d8  00000000004011d8  000011d8  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 11 .rela.dyn     00000048  0000000000401208  0000000000401208  00001208  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 12 .rela.plt     000000a8  0000000000401250  0000000000401250  00001250  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 13 .rodata       000000de  00000000004012f8  00000000004012f8  000012f8  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 14 .eh_frame_hdr 0000003c  00000000004013d8  00000000004013d8  000013d8  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 15 .eh_frame     000000cc  0000000000401418  0000000000401418  00001418  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 16 .note.gnu.property 00000040  00000000004014e8  00000000004014e8  000014e8  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 17 .note.ABI-tag 00000020  0000000000401528  0000000000401528  00001528  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 18 .init_array   00000008  0000000000402df8  0000000000402df8  00001df8  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 19 .fini_array   00000008  0000000000402e00  0000000000402e00  00001e00  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 20 .dynamic      000001d0  0000000000402e08  0000000000402e08  00001e08  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 21 .got          00000010  0000000000402fd8  0000000000402fd8  00001fd8  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 22 .got.plt      00000050  0000000000402fe8  0000000000402fe8  00001fe8  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 23 .data         00000004  0000000000403038  0000000000403038  00002038  2**0
                  CONTENTS, ALLOC, LOAD, DATA
```

```
 23 .data         00000004  0000000000403038  0000000000403038  00002038  2**0
                  CONTENTS, ALLOC, LOAD, DATA
 24 .bss          00000010  0000000000403040  0000000000403040  0000203c  2**5
                  ALLOC
 25 .comment      0000002e  0000000000000000  0000000000000000  0000203c  2**0
                  CONTENTS, READONLY
 26 .annobin.notes 0000014f  0000000000000000  0000000000000000  0000206a  2**0
                  CONTENTS, READONLY
 27 .gnu.build.attributes 00000144  0000000000405050  0000000000405050  000021bc  2**2
                  CONTENTS, READONLY, OCTETS
SIZE INFO
   text    data     bss     dec     hex filename
   2433     580      16    3029     bd5 md_mem_100
```

Analizēju izmēru md_mem_100 ar rīkiem objdump un size. Var redzēt, ka **TEXT** segments aizņem 2433 baitus, **DATA** segments 580 baitus, bet **BSS** segments 16 baitus. TEXT segments satur programmas izpildāmo kodu, DATA segments satur inicializētos globālos vai statiskos mainīgos, bet BSS segments satur neinicializētos globālos vai statiskos mainīgos. Tā kā rezervēšanas metode tiek padota caur komandrindas argumentu un šī atmiņa tiek rezervēta programmas palaišanas brīdi, tad var secināt, ka šī atmiņa nav atkarīga no izmantotās dinamiskās atmiņas rezervācijas veida.